Основе програмирања и програмски језици

1 Вежбе 2: Управљање подацима у Python-у

1.1 Текстуални тип података

Текстуални тип података - стрингови у Python-у се користе за чување и манипулацију текстом, а пишу се унутар једноструких ('text') или двоструких ("text") наводника. Оба начина функционишу потпуно исто.

Листинг 1: Текстуални тип података

```
print("Zdravo")  # Dvostruki navodnici
print('Zdravo')  # Jednostruki navodnici
```

Могу се користити наводници унутар стринга, под условом да се они не поклапају са спољашњим наводницима који ограничавају тај стринг.

Листинг 2: Наводници у стрингу

```
print("To je 'u redu'")        # Unutra su jednostruki, spolja dvostruki
print('Zovu ga "Dzoni"')       # Unutra su dvostruki, spolja jednostruki
```

Ако желите да доделите текст који се протеже кроз више редова, користите три наводника (било двострука или једнострука). Преломи редова ће остати на истом месту као и у коду.

Листинг 3: Вишелинијски стринг

```
tekst = """Ovo je string
koji se proteze
kroz nekoliko redova."""
print(tekst)
```

Као и у многим другим језицима, стрингови у Python-у су заправо низови карактера. Пытхон нема посебан тип података за само једно слово (карактер) – једно слово је једноставно стринг дужине 1. Елементима стринга приступате помоћу угластих заграда [], имајући на уму да индексирање почиње од 0.

Листинг 4: Испис делова стринга

```
poruka = "Zdravo!"
print(poruka[0])  # Ispisuje: Z (prvo slovo)
print(poruka[1])  # Ispisuje: d (drugo slovo)
```

Пошто је стринг низ, кроз његова слова можемо проћи и помоћу for петље.

Листинг 5: Испис слова преко петље

```
for slovo in "banana":
    print(slovo)  # Ispisuje svako slovo u novom redu
```

Да бисмо сазнали колико стринг има карактера (укључујући размаке и знаке узвика), користите функцију/метод len().

Листинг 6: Дужина стринга

```
a = "Zdravo, svete!"
print(len(a))  # Ispisuje: 14
```

Python омогућава веома лаку проверу да ли се одређена реч или слово налазе унутар неког текста. Опција in– враћа True ако је текст присутан a not in – враћа True ако текст није присутан.

Листинг 7: Провера садржаја стринга

```
tekst = "Najbolje stvari u zivotu su besplatne!"
# Provera sa 'in'
print("besplatne" in tekst)  # Ispisuje: True

# Provera sa 'not in'
print("skupo" not in tekst)  # Ispisuje: True
```

Помоћу синтаксе за исецање (slice) можемо издвојити и добити одређени опсег карактера из неког стринга. Да бисмо добили део стринга, потребно је да наведемо почетни индекс и крајњи индекс, раздвојене двотачком. Први карактер у стрингу увек има индекс 0.

Листинг 8: Исецање стринга

```
b = "Hello, World!"
print(b[2:5])  # Izdvaja karaktere od pozicije 2 do pozicije 5 (koja nije ukljucena)
```

Ако изоставите почетни индекс, опсег за исецање ће аутомацки почети од првог карактера у стрингу.

Листинг 9: Исецање стринга

```
b = "Hello, World!"
print(b[:5])  # Izdvaja karaktere od samog pocetka do pozicije 5 (koja nije ukljucena)
```

Ако изоставимо крајњи индекс, опсег за исецање ће ићи све до самог краја стринга.

Листинг 10: Исецање стринга

```
b = "Hello, World!"
print(b[2:])  # Izdvaja karaktere pocevsi od pozicije 2 pa sve do kraja stringa
```

За покретање исецања од краја стринга можемо искористити негативне индексе. На пример, индекс -1 означава последњи карактер, -2 претпоследњи и тако даље.

Листинг 11: Исецање стринга

```
b = "Hello, World!"
print(b[-5:-2])  # Izdvaja od slova "o" (pozicija -5) do slova "d" (pozicija -2, koja nije
    ukljucena)
```

Python има скуп уграђених метода које можете користити над стринговима како бисте изменили њихов изглед. Важно је запамтити да су стрингови у Python-у непроменљиви (immutable), што значи да ове методе не мењају оригинални стринг, већ увек враћају нови, модификовани стринг. Метода upper()претвара сва слова у текстуалном податку у велика слова.

Листинг 12: Конвертовање у велика слова

```
a = "Hello, World!"
print(a.upper())  # Ispisuje: "HELLO, WORLD!"
```

Метода lower()претвара сва слова у текстуалном податку у мала слова.

Листинг 13: Конверзија у мала слова

```
a = "Hello, World!"
print(a.lower())  # Ispisuje: "hello, world!"
```

Белине или празни простори (whitespace) пре почетка текста или након његовог краја се често појављују грешком. Метода strip() уклања све те празне просторе са почетка и краја стринга.

Листинг 14: Уклањање празнина

```
a = " Hello, World! "
print(a.strip())  # Ispisuje: "Hello, World!"
```

Метода replace() замењује одређени карактер или цео подстринг неким другим текстом који ви проследите као параметар.

Листинг 15: Замена карактера

```
a = "Hello, World!"
print(a.replace("H", "J"))  # Ispisuje: "Jello, World!"
```

Метода split() дели стринг на листу подстрингова уколико пронађе инстанце сепаратора који смо дефинисали. На пример, ако стринг поделимо преко зареза, добићемо делове текста који су били раздвојени тим знаком.

Листинг 16: Раздвајање стринга

```
a = "Hello, World!"
print(a.split(","))  # Ispisuje listu: ['Hello', ' World!']
```

Конкатенација означава процес спајања или комбиновања два или више текстуалних података у један јединствен стринг. У Python-у се за ову операцију најчешће користи оператор плус (+). Да бисмо спојили две текстуалне променљиве у једну, довољно је да између њих поставимо оператор +.

Листинг 17: Спајање стрингова

```
a = "Hello"
b = "World"
c = a + b
print(c)  # Ispisuje: "HelloWorld"
```

Као што видимо у претходном примеру, оператор + само спаја текст без додавања размака између њих. Да бисмо додали празан простор (размак) између две променљиве, потребно је да експлицитно убацимо стринг који садржи размак .

Листинг 18: Спајање стрингова и размак

```
a = "Hello"
b = "World"
c = a + " " + b
print(c)  # Ispisuje: "Hello World"
```

Важно је напоменути да у Python-у не можемо директно помоћу оператора + спојити стринг и број. Уколико то покушамо, Python ће пријавити грешку (TypeError). За комбиновање бројева и текста користе се напредније методе попут форматирања стрингова.

Листинг 19: Додавање броја у стринг

```
godine = 36
tekst = f"Moje ime je Dzon i imam {godine} godina"
print(tekst)  # Ispisuje: "Moje ime je Dzon i imam 36 godina"
```

Витичасте заграде унутар ф-стринга нису резервисане само за променљиве. Оне могу да приме и различите модификаторе, резервисана места, па чак и целе операције или позиве функција. Можемо, на пример, извршити математичку операцију директно унутар текста.

Листинг 20: Комбиновање операција у стрингу

```
stranica = 5
tekst = f"Povrsina kvadrata je {stranica*stranica} "
print(tekst)  # Ispisuje: "Povrsina kvadrata je 25 "
```

Такође, унутар витичастих заграда можемо додати модификаторе формата који се одвајају двотачком. На пример, модификатор :.2f говори Python-у да прикаже број са покретним зарезом фиксно на две децимале.

Листинг 21: Приказ децималних места

```
cena = 59
tekst = f"Cena artikla je {cena:.2f} dolara"
print(tekst)  # Ispisuje: "Cena artikla je 59.00 dolara"
```

Да бисмо у стринг убацили карактере које иначе не можемо директно да откуцамо (попут наводника који би прекинули стринг или команде за нови ред), користи се специјални знак за "бекство"– коса црта уназад (\). Овај знак говори Python-у да карактер који следи одмах иза њега има посебно значење и да не треба да се тумачи као обичан текст или синтаксни прекид. Један од најчешћих примера је убацивање дуплих наводника унутар стринга који је већ окружен дуплим наводницима. Коришћењем саопштавамо Python-у да желимо буквалан испис наводника, а не крај стринга. Поред решавања проблема са наводницима, коса црта уназад у комбинацији са одређеним словима креира команде за форматирање текста. Најкоришћенија је комбинација $\backslash n$ која пребацује сав текст који следи иза ње у потпуно нови ред.

Листинг 22: Нови ред

```
tekst = "Zdravo\nSvemir!"
print(tekst)
```

Уколико нам је потребно да направимо већи хоризонтални размак у тексту, односно да симулирамо притисак тастера Tab на тастатури, користи се специјална секвенца $\backslash t$.

Листинг 23: Таб

```
tekst = "Ime:\tPetar"
print(tekst)  # Ispisuje: Ime:    Petar
```

Python поседује богат скуп уграђених метода/функција које можемо позвати над текстуалним подацима за потребе претраге, анализе и трансформације. Све методе стрингова увек враћају нову вредност, остављајући оригинални стринг непромењеним.
Метода capitalize() претвара само прво слово целог стринга у велико слово, док сва остала слова у тексту постају мала.

Листинг 24: Прво велико слово

```
tekst = "python je sjajan."
print(tekst.capitalize())  # Ispisuje: "Python je sjajan."
```

Метода count()броји колико се пута одређена реч, подстринг или карактер појављује унутар текста који анализирамо.

Листинг 25: Бројање инстанци стринга

```
tekst = "Volim jabuke, jabuke su moje omiljeno voce."
print(tekst.count("jabuke"))  # Ispisuje: 2
```

Метода find() претражује стринг и тражи прву појаву прослеђене речи или карактера. Ако га пронађе, враћа индекс (позицију) на којој тај подстринг почиње. Уколико вредност не постоји у стрингу, метода враћа -1.

Листинг 26: Провера позиције инстанце претраге

```
tekst = "Zdravo, dobrodosli u svet Pythona."
print(tekst.find("dobrodosli"))  # Ispisuje indeks pocetka reci: 8
```

Метода startswith()проверава да ли стринг почиње одређеном вредно71у и на основу тога враћа тачно (True) или нетачно (False). Слично њој, метода endswith()врши исту проверу за крај стринга.

Листинг 27: Провера почетка и краја стринга

```
tekst = "Python je programski jezik."
print(tekst.startswith("Python"))  # Ispisuje: True
print(tekst.endswith("jezik"))     # Ispisuje: False (jer string zavrsava tackom)
```

Метода isalnum()проверава да ли се текст састоји искључиво од алфанумеричких карактера, што значи да стринг мора садржати само слова и бројеве, без размака, зареза или специјалних знакова.

Листинг 28: Провера алфанумеричких знакова

```
korisnicko_ime = "Programer123"
print(korisnicko_ime.isalnum())  # Ispisuje: True
```

Задатак 1.

Шта ће исписати следећи код дат у Листингу 29 ако корисник унесе своје име малим словима (нпр. milos)?

Листинг 29: Листинг уз Задатак 1.

```
ime = input("Unesite ime: ")
poruka = f"Korisnik {ime.capitalize()} ima {len(ime)} slova u imenu."
print(poruka)
```

Задатак 2.

Шта је резултат исецања (slicing) над стрингом b = "Python_Programer"у дата три случја?

а) print(b[7:14])

б) print(b[:6])

в) print(b[-3:])

1.2 Оператори

Python оператори се користе за извршавање операција над променљивим и вредностима.

У Табели 1 су дати примери аритметичких оператора.

Табела 1: Аритметички оператори

Оператор	**Назив**	**Пример**	**Објашњење**
+	Сабирање	$x + y$	Сабира две вредности
−	Одузимање	$x - y$	Одузима десну вредност од леве
*	Множење	$x * y$	Множи две вредности
/	Дељење	x/y	Дели леву вредност десном (резултат је float)
%	Модул	$x\%y$	Враћа остатак при дељењу
**	Степеновање	$x * *y$	Подиже број x на степен y
//	Целобројно дељење	$x//y$	Дели бројеве и заокружује на цео број

У Табели 2 су дати примери оператора доделе.

Табела 2: Оператори доделе

Оператор	**Пример**	**Једнако је изразу**
$=$	$x = 5$	$x = 5$
$+=$	$x+ = 3$	$x = x + 3$
$-=$	$x- = 3$	$x = x - 3$
$*=$	$x* = 3$	$x = x * 3$
$/=$	$x/ = 3$	$x = x/3$
$\%=$	$x\% = 3$	$x = x\%3$
$//=$	$x// = 3$	$x = x//3$
$**=$	$x** = 3$	$x = x**3$

У Табели 3 су дати примери оператора поређења.

Табела 3: Оператори поређења

Оператор	**Назив**	**Пример**
$==$	Једнако	$x == y$
$!=$	Није једнако	$x! = y$
$>$	Веће од	$x > y$
$<$	Мање од	$x < y$
$>=$	Веће или једнако	$x >= y$
$<=$	Мање или једнако	$x <= y$

У Табели 4 су дати примери логичких оператора.

Табела 4: Логички оператори

Оператор	**Опис**	**Пример**
and	Враћа True ако су оба исказа тачна	$x < 5$ *and* $x < 10$
or	Враћа True ако је бар један исказ тачан	$x < 5$ *or* $x < 4$
not	Обрће резултат (враћа False ако је тачан)	*not*$(x < 5$ *and* $x < 10)$

У Табели 5 су дати примери оператора идентитета.

Табела 5: Оператори идентитета

Оператор	**Опис**	**Пример**
is	Враћа True ако су променљиве исти објекат	*x is y*
is not	Враћа True ако променљиве нису исти објекат	*x is not y*

У Табели 6 су дати примери оператора чланства.

Табела 6: Оператори чланства

Оператор	**Опис**	**Пример**
in	Враћа True ако је вредност присутна у објекту	*x in y*
not in	Враћа True ако вредност није присутна	*x not in y*

1.3 Логички тип података

Логички тип података (булови - Boolean) представљају једну од две вредности у логици и програмирању: True(Тачно) или False(Нетачно). Они су основа за доношење одлука у коду, јер нам омогућавају да сазнамо да ли је неки израз тачан или не. Када упоредимо две вредности помоћу оператора, Python ће аутомацки израчунати израз и вратити логички тип одговора.

Листинг 30: Логички тип

```
print(10 > 9)   # Ispisuje: True
print(10 == 9)  # Ispisuje: False
print(10 < 9)   # Ispisuje: False
```

Логичке вредности се најчешће користе када постављамо услов унутар if наредбе. Python тада проверава да ли је услов тачан или нетачан и на основу тога одлучује који део кода ће се извршити.

Листинг 31: if

```
a = 200
b = 33
if b > a:
print("b je vece od a")
else:
print("b nije vece od a")  # Ovaj uslov ce se ispuniti
```

Уграђена функција bool() омогућава нам да проверимо логичку вредност било ког податка или промењиве.

Листинг 32: bool()

```
x = "Hello"
y = 15
print(bool(x))  # Ispisuje: True
print(bool(y))  # Ispisuje: True
```

У Python-у је скоро свака вредност True уколико има било какав садржај. Било који стринг је True осим празног стринга. Било који број је True осим нуле. Било која листа, торка, сет или речник су True осим оних који су потпуно празни.

Листинг 33: True вредности

```
print(bool("abc"))                          # Ispisuje: True
print(bool(123))                            # Ispisuje: True
print(bool(["apple", "cherry", "banana"])) # Ispisuje: True
```

Вредности које се претварају у False су малобројне. То су искључиво празни објекти и колекције попут (), {},"", број 0, вредност None, и наравно, сама логичка вредност False . Такође, False враћају и објекти направљени од класа чија функција len враћа 0 или False .

Листинг 34: False вредности

```
print(bool(False)) # Ispisuje: False
print(bool(None))  # Ispisuje: False
print(bool(0))     # Ispisuje: False
print(bool(""))    # Ispisuje: False
print(bool([]))    # Ispisuje: False
```

Функције које сами напишемо такође могу враћати логичку вредност. То нам омогућава да извршавамо условне блокове кода директно на основу резултата функције.

Листинг 35: Основне операције

```
def mojaFunkcija():
    return True
if mojaFunkcija():
    print("DA!")  # Izvrsava se jer funkcija vraca True
else:
    print("NE!")
```

Поред тога, Python има много уграђених функција које враћају логичку вредност, као што је isinstance() , која служи за проверу да ли неки објекат припада одређеном типу података.

Листинг 36: Основне операције

```
x = 200
print(isinstance(x, int))  # Ispisuje: True (jer je x ceo broj)
```

Задатак 3.

Шта ће исписати дате наредбе у Листингу 37 за проверу логичких вредности помоћу функције bool()?

Листинг 37: Листинг уз Задатак 3.

```
print(bool("abc"))
print(bool(""))
print(bool(10 - 10))
```

1.4 Листе

Python листе служе за складиштење више ставки у једној променљивој. Оне су један од уграђених типова података који се користе за чување колекција података. Листе се креирају помоћу угластих заграда [].

Листинг 38: Листа

```
voce_lista = ["jabuka", "banana", "visnja"]
```

Ставке у листи имају три кључне особине: уређене су, променљиве су и дозвољавају дупликате. Поред тога, све ставке су индексиране – прва ставка има индекс [0], друга [1], и тако даље.

Листинг 39: Елемент листе

```
voce_lista = ["jabuka", "banana", "visnja"]
print(voce_lista[0]) # rezultat: jabuka
```

Уређеност се односи на то да ставке имају тачно дефинисан редослед који се не мења. Нови елементи се увек додају на сам крај листе. Променљивост се односи на то да се елементи могу мењати, додавати или брисати након што је листа већ направљена. Дозвољавају дупликате пошто свака ставка има свој индекс, листа може да садржи више истих вредности. Да бисте израчунали колико ставки има у листи, користи се уграђена функција len() .

Листинг 40: Дужина листе - број елемената листе

```
voce_lista = ["jabuka", "banana", "visnja"]
print(len(voce_lista)) # rezultat: 3
```

Ставке у листи могу бити било ког типа података. Једна листа може да комбинује различите типове података једном.

Листинг 41: Разни типови елемената листе

```
list1 = ["abc", 34, True, 40, "male"]
```

Из угла Python-а, листе су објекти са типом податка 'list' . За креирање листе можете употребити и експлицитан конструктор list().

Листинг 42: Креирање листе преко list()

```
thislist = list(("jabuka", "banana", "visnja")) # obratiti paznju na duple zagrade ()
print(thislist)
```

Ставкама листе се приступоа преко њихових индексних бројева унутар угластих заграда. Први индекс је 0. Негативно индексирање значи да се креће од краја листе.

Листинг 43: Приступање елементима листе

```
thislist = ["jabuka", "banana", "visnja"]
print(thislist[0]) # rezultat: jabuka
print(thislist[-1]) # rezultat: visnja
print(thislist[-2]) # rezultat: banana
print(thislist[1]) # rezultat: banana
```

Може се дефинисати интервал индекса које желимо издвојити преко :. Када се дефинише интервал резултат је нова листа са одабраним елементима. Елемент дефинисан другим индексом интервала се не додаје, до њега, не укључујући њега.

Листинг 44: Приступање елементима листе

```
thislist = ["jabuka", "banana", "visnja","kruska", "jagoda"]
print(thislist[1:3]) #rezultat ["banana", "visnja"]
```

Ако се изостави први индекс у интервалу, интервал креће од почетка листе. Ако се изостави други индекс интервала издвајају се елементи до краја листе.

Листинг 45: Приступање елементима листе

```
thislist = ["jabuka", "banana", "visnja","kruska", "jagoda"]
print(thislist[:3]) #rezultat [jabuka", "banana", "visnja"]
print(thislist[1:]) #rezultat ["banana", "visnja","kruska", "jagoda"]
```

Промена елемента листе се обавља додељивањем нове вредности преко индекса елемента.

Листинг 46: Измена елемената листе

```
thislist = ["jabuka", "banana", "visnja","kruska", "jagoda"]
thislist[0] = "mandarina"
print(thelist) #rezultat: ["mandarina", "banana", "visnja","kruska", "jagoda"]
```

Измена неколико вредности се може изменити коришћењем интервала индекса.

Листинг 47: Измена елемената листе

```
thislist = ["jabuka", "banana", "visnja","kruska", "jagoda"]
thislist[1:3] = ["mandarina","borovnica"]
print(thelist) #rezultat: ["jabuka", "mandarina", "borovnica","kruska", "jagoda"]
thislist[1:3] = ["mandarina"]
print(thelist) #rezultat: ["jabuka", "mandarina", "mandarina","kruska", "jagoda"]
```

Уметање елемента се може обавити функцијом *insert*().

Листинг 48: Уметање елемената листе

```
thislist = ["jabuka", "banana", "visnja"]
thislist.insert(1,"mandarina")
print(thelist) #rezultat: ["jabuka", "mandarina", "banana","visnja"]
```

Да би се додао елемент на крај листе можемо користити опцију *append*().

Листинг 49: Додавање елемената листи

```
thislist = ["jabuka", "banana", "visnja"]
thislist.append("mandarina")
print(thelist) #rezultat: ["jabuka", "banana", "visnja","mandarina"]
```

Да се додају елементи друге листе можемо користити *extend*().

Листинг 50: Додавање листе на листу

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple", "papaya"]
thislist.extend(tropical)
print(thislist)
```

Укљањање елемента листе се обавља помоћу *remove*(). Укљања се прва инстанца наведене вредности не и остале ако има дупликата.

Листинг 51: Уклањање елемената листе

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

Да би се уклонио елемент конкретног индекса користимо *pop*(). Ако се не унесе индекс подразумевано уклања последњи елемент листе.

Листинг 52: Уклањање елемената листе

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

Брисање елемената се може урадити и кључном речи *del* као и брисање целе листе.

Листинг 53: Уклањање елемената листе

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)

thislist = ["apple", "banana", "cherry"]
del thislist
```

Брисање/чи71ење листе са укљањањем елемената али остављањем празне листе се може урадити са *clear*().

Листинг 54: Уклањање елемената листе

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

Сортирање елемената листе се може урадити са *sort*(). Овај метод је осетљив на велика и мала слова па прво сортира велика слова.

Листинг 55: Сортирање елемената листе

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist) #sortiranje alfanumericki

thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist) #numericko sortiranje

thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True) # sortiranje u suprotnom smeru - descending
print(thislist)
```

Ако се само желе обрнути редослед елемената може се корисити *reverse*().

Листинг 56: Сортирање елемената листе

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

Листа се не може копирати коришћењем оператора доделе већ можемо користити *copy*(), преко *list*() и оператором :.

Листинг 57: Копирање листе

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)

thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)

thislist = ["apple", "banana", "cherry"]
mylist = thislist[:]
print(mylist)
```

Спајање - конкатенација листи се може урадити на неколико начина.

Листинг 58: Спајање листи

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)

list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```

1.5 Торке

Торке (Tuples) се користе за складиштење више ставки у једној променљивој. Торка је један од уграђених типова података у Python-у који служе за чување колекција података. Торка је колекција која је уређена и неизменљива. Пише се са обичним заградама. Уређене су, неизменљиве и дозвољавају дупликате. Индексиране су, тако да прва ставка има индекс [0], друга [1] итд. Ставке имају дефинисан редослед и тај редослед се неће променити. Не можемо мењати, додавати или уклањати ставке након што је торка креирана. Пошто су индексиране, могу имати више ставки са истом вредношћу.

Листинг 59: Торка

```
podaci = ("apple", "banana", "cherry")
print(thistuple)

thistuple = "apple", "banana", "cherry"
print(thistuple)
```

Да се одреди колико елемената има користи се функција *len*().

Листинг 60: Број елемената торке

```
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Креирање торке са једном ставком мора се обавити стављањем зареза након ставке, иначе Python неће препознати да је у питању торка.

Листинг 61: Креирање торке са једном ставком

```
thistuple = ("apple",)
print(type(thistuple))

# nije torka
thistuple = ("apple")
print(type(thistuple))

#prazna torka
thistuple = ()
print(type(thistuple))
```

Торка такође може садржати елементе различитих типова. Торка се може креирати и коришћењем функције *tuple*().

Листинг 62: Креирање торке

```
thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Ставкама торке се приступа преко њиховог индексног броја унутар угластих заграда []. Дозвољено је и негативно индексирање (бројање од краја), као и узимање опсега (исечака/слицинг).

Листинг 63: Приступ ставкама/елементима торке

```
# Primer 1: Pristup preko pozitivnog indeksa (ispisuje drugu stavku "banana")
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])

# Primer 2: Negativno indeksiranje (-1 je poslednja stavka "cherry")
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])

# Primer 3: Opseg indeksa (Slicing) - vraca novu torku sa indeksima 2, 3 i 4
# Napomena: Indeks 5 nije ukljucen u rezultat
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
```

Пошто су торке неизменљиве (immutable), не можемо директно додавати, мењати или брисати њихове елементе након креирања. Заобилазно решење је конвертовање торке у листу, измена листе, и поновно враћање у торку.

Листинг 64: Промена вредности елемента торке

```
# Primer 1: Promena vrednosti stavke zaobilaznim putem
x = ("apple", "banana", "cherry")
y = list(x)         # Prebacivanje u listu
y[1] = "kiwi"       # Izmena elementa
x = tuple(y)        # Vracanje u torku
print(x)

# Primer 2: Dodavanje elemenata (Spajanje torke sa drugom torkom)
# Kada dodajete torku sa jednom stavkom, obavezan je zarez na kraju!
```

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y
print(thistuple)

# Primer 3: Potpuno brisanje torke iz memorije pomocu "del" kljucne reci
thistuple = ("apple", "banana", "cherry")
del thistuple
# print(thistuple) -> Ovo bi izbacilo gresku (NameError) jer torka vise ne postoji!
```

Додавање и брисање се обавља заобилазно преко листи.

Листинг 65: Додавање/брисање елемената торке

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)


thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y

print(thistuple)

 thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)

thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error because the tuple no longer exists
```

Када креирамо торку и доделимо јој вредности, то се зове "паковање" (пацкинг). Извачење тих вредности назад у појединачне промењиве назива се "отпакивање" (унпацкинг). Ако је број променљивих мањи од броја ставки, користи се звездица $*$ да покупи преостале вредности у листу.

Листинг 66: Отпакивање торке

```
# Primer 1: Standardno otpakivanje torke
fruits = ("apple", "banana", "cherry")
(green, yellow, red) = fruits
print(green)
print(yellow)
print(red)

# Primer 2: Koriscenje zvezdice (*) kada ima vise stavki nego promenljivih
# Preostale stavke se pakuju u listu dodeljenu promenljivoj "red"
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
(green, yellow, *red) = fruits
print(green)
print(yellow)
print(red)  # Ispisuje: ['cherry', 'strawberry', 'raspberry']
```

Две или више торки можете спојити у једну нову користећи оператор плус $+$, док оператор $*$ служи да умножи садржај торке задати број пута.

Листинг 67: Спајање торки

```
# Primer 1: Spajanje dve torke u jednu novu pomocu "+" operatora
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
print(tuple3)

# Primer 2: Multiplikacija (umnozavanje) sadrzaja torke pomocu "*" operatora
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
print(mytuple)  # Ispisuje sve stavke dva puta unutar jedne torke
```

Python има две уграђене методе које можемо извршити над торкама: број појављивања одређене вредности (count()) и проналази прву позицију (индекс) задате вредности(index()).

Листинг 68: Методе за торке

```
# Primer 1: Metoda count() - broji koliko puta se pojavljuje broj 5
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)  # Rezultat je 2

# Primer 2: Metoda index() - trazi na kojoj je poziciji prvo pojavljivanje broja 8
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.index(8)
print(x)  # Rezultat je 3 (indeks cetvrte pozicije)
```

Задатак 4.

Дата је листа brojevi = [10, 20, 30]. Напишите наредбе које:

- Додају број 40 на крај листе
- Умећу број 15 на позицију са индексом 1
- Бришу број 20 из листе

Задатак 5.

Имамо торку torka = (”crvena”, ”plava”, ”zelena”). Покушано је извршење кода torka[1] = ”zuta”. Шта ће се десити и зашто?

1.6 Сетови/Скупови

Скупови/Сетови (set) се користе за складиштење више ставки у једној променљивој. Скуп је колекција која је неуређена, неизменљива и неиндексирана. Пише се са витичастим заградама . Ставке немају дефинисан редослед. Могу се појавити у другачијем редоследу сваки пут и не можете им приступити преко индекса или кључа. Саме ставке се не могу мењати након што је скуп креиран, али можете слободно брисати старе и додавати нове ставке. Скупови не могу имати две ставке са истом вредношћу (дупликати се игноришу). Вредности True и 1 се третирају као иста вредност, као што се и False и 0 третирају као дупликати.

Листинг 69: Скуп

```
# Primer: Kreiranje skupa (duplikat "apple" ce biti ignorisan)
thisset = {"apple", "banana", "cherry", "apple"}
print(thisset)
```

Пошто скупови немају индексе, ставкама не можете приступити помоћу [0] или сличних позиција. Уместо тога, кроз скуп пролазите помоћу *for* петље или проверавате да ли је вредност присутна користећи кључну реч *in*.

Листинг 70: Претрага елемената скупа

```
# Prolazak kroz skup pomocu for petlje
thisset = {"apple", "banana", "cherry"}
for x in thisset:
  print(x)

# Provera da li se "banana" nalazi u skupu (Vraca True ili False)
thisset = {"apple", "banana", "cherry"}
print("banana" in thisset)
```

Када је скуп креиран, не можете мењати постојеће ставке, али можете додати нове помоћу методе *add*(). Ако желите да додате ставке из друге колекције (други скуп, листа, торка), користите методу *update*().

Листинг 71: Додавање елемената скупа

```
# Primer 1: Dodavanje jedne stavke pomocu add()
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)

# Primer 2: Dodavanje elemenata iz drugog skupa pomocu update()
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}
thisset.update(tropical)
print(thisset)
```

За брисање ставке користе се методе *remove*() или *discard*(). Главна разлика је у томе што *remove*() избацује грешку ако ставка не постоји, док *discard*() неће избацити грешку. Метода *pop*() брише насумичну ставку јер скуп нема редослед.

Листинг 72: Мењање елемената скупа

```
# Primer 1: Koriscenje remove() (izbacuje gresku ako objekat ne postoji)
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)

# Primer 2: Koriscenje discard() (NE izbacuje gresku ako objekat ne postoji)
thisset = {"apple", "banana", "cherry"}
thisset.discard("banana")
print(thisset)

# Primer 3: Koriscenje pop() (brise nasumicnu stavku i vraca njenu vrednost)
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x) # Prikazuje izbrisanu stavku
print(thisset) # Prikazuje ostatak skupa

# Primer 4: clear() prazni ceo skup, dok del potpuno brise skup iz memorije
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset) # Prikazuje set() - prazan skup
```

Python пружа неколико начина за спајање скупова. Можемо користити *union*() или оператор да добијемо све елементе из оба скупа, или *intersection*() односно оператор да задржимо само заједничке елементе.

Листинг 73: Операције над скуповима

```
# Primer 1: union() vraca novi skup sa svim stavkama iz oba skupa
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
set3 = set1.union(set2)
print(set3)

# Primer 2: intersection() vraca samo stavke koje postoje u oba skupa
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
set3 = set1.intersection(set2)
print(set3) # Ispisuje samo: {"apple"}

# Primer 3: difference() vraca stavke iz prvog skupa koje NE postoje u drugom skupu
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}
set3 = set1.difference(set2)
print(set3) # Ispisuje: {"banana", "cherry"}
```

1.7 Речници

Речници (Dictionaries) се користе за складиштење вредности у паровима кључ:вредност (*key* : *value*). Речник је колекција која је уређена, изменљива и не дозвољава дупликате кључева. Пише се са витичастим заградама {} и има кључеве и вредности. Од верзије Python 3.7, речници су уређени. То значи да ставке имају дефинисан редослед и тај редослед се неће променити. Можемо мењати, додавати или уклањати ставке након што је речник креиран. Речници не могу имати два кључа са истим именом. Ако наведете дупли кључ, нова вредност ће пребрисати стару.

Листинг 74: Речник

```
# Primer: Kreiranje recnika (duplirani kljuc "year" ce prepisati prvu vrednost)
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(thisdict) # Ispisuje 'year': 2020
print(len(thisdict)) # Funkcija len() vraca broj stavki (parova) u recniku
```

Ставкама речника можете приступити тако што ћете унутар угластих заграда [] навести име кључа, или коришћењем методе *get*(). Такође можете извући све кључеве помоћу *keys*(), све вредности помоћу *values*(), или парове као торке преко методе *items*().

Листинг 75: Приступ ставкама речника

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: Pristup vrednosti preko kljuca unutar []
x = thisdict["model"]

# Primer 2: Pristup vrednosti preko metode get()
x = thisdict.get("model")

```

```
# Primer 3: Dobijanje liste svih kljuceva
kljucevi = thisdict.keys()

# Primer 4: Dobijanje liste svih vrednosti
vrednosti = thisdict.values()

# Primer 5: Dobijanje parova kljuc-vrednost u obliku torki
parovi = thisdict.items() # Vraca [('brand', 'Ford'), ('model', 'Mustang'), ...]
```

Вредност специфичне ставке можете променити тако што ћете се референцирати на њено име кључа, или коришћењем методе *update*().

Листинг 76: Измена речника

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: Direktan upis nove vrednosti preko kljuca
thisdict["year"] = 2018

# Primer 2: Promena vrednosti pomocu metode update()
thisdict.update({"year": 2020})
```

Додавање нове ставке у речник се врши кориш]-ћењем новог кључа и додељивањем вредности, или преко методе *update*().

Листинг 77: Додавање нове ставке речника

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: Dodavanje novog para kljuc-vrednost direktno
thisdict["color"] = "red"

# Primer 2: Dodavanje novog para preko metode update()
thisdict.update({"engine": "V8"})
```

Постоји неколико уграђених метода за уклањање ставки из речника: *pop*() уклања ставку са одређеним именом кључа, *popitem*() уклања последњу убачену ставку, а *clear*() празни цео речник.

Листинг 78: Уклањање ставки речника

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: pop() uklanja stavku sa tacno odredjenim kljucem
thisdict.pop("model")

# Primer 2: popitem() uklanja poslednju ubacenu stavku
thisdict.popitem()

# Primer 3: del kljucna rec brise stavku ili potpuno brise ceo recnik
del thisdict["brand"]

# Primer 4: clear() prazni recnik, ali struktura ostaje (postaje prazan {})
thisdict.clear()
```

Можете пролазити кроз речник користећи *for* петљу. Када пролазите кроз речник, подразумевано се враћају кључеви, али постоје методе којима експлицитно тражите вредности или парове.

Листинг 79: Приказ ставки речника

```python
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: Ispis svih KLJUCEVA jedan po jedan
for x in thisdict:
  print(x)

# Primer 2: Ispis svih VREDNOSTI preko kljuceva
for x in thisdict:
  print(thisdict[x])

# Primer 3: Ispis svih VREDNOSTI direktno pomocu .values() metode
for x in thisdict.values():
  print(x)

# Primer 4: Ispis i KLJUCEVA i VREDNOSTI istovremeno pomocu .items()
for x, y in thisdict.items():
  print(x, y)
```

Не можемо копирати речник једноставним уписивањем $dict2 = dict1$, јер ће $dict2$ бити само референца (пречица) ка $dict1$, па ће се све измене у првом видети и у другом. Уместо тога, користе се методе $copy()$ или уграђена функција $dict()$.

Листинг 80: Копирање речника

```python
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Primer 1: Pravljenje kopije pomocu metode copy()
mydict = thisdict.copy()

# Primer 2: Pravljenje kopije pomocu funkcije dict()
mydict = dict(thisdict)
```

Речник може да садржи друге речнике унутар себе. Ово се назива угњеждени речници (nested dictionaries).

Листинг 81: Угњеждени речник

```python
# Primer 1: Jedan veliki recnik koji sadrzi tri recnika unutar sebe
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}

# Primer 2: Pristup stavkama u ugnjezdenom recniku (ispisuje ime drugog deteta: Tobias)
print(myfamily["child2"]["name"])
```

Поред стандардних метода за мењање и брисање, речници имају листу уграђених функција које олаксавају рад.

Листинг 82: Методе за речнике

```python
# Primer 1: setdefault() - Vrada vrednost kljuda ako postoji.
# Ako ne postoji, ubacuje kljuc sa zadatom vrednoSCu.
car = {"brand": "Ford", "model": "Mustang"}
x = car.setdefault("color", "white")
print(car) # Sada sadrZi i 'color': 'white'

# Primer 2: fromkeys() - Kreira recnik sa zadatim kljucevima i istom pocetnom vrednoscu
x = ('key1', 'key2', 'key3')
y = 0
thisdict = dict.fromkeys(x, y)
print(thisdict) # Ispisuje: {'key1': 0, 'key2': 0, 'key3': 0}
```

Задатак 6.

Шта је излаз кода датог у Листингу 83 који користи речник?

Листинг 83: Листинг уз Задатак 6.

```python
student = {"ime": "Ana", "indeks": 123, "indeks": 456}
print(student["indeks"])
print(len(student))
```

2 Самосталне вежбе 2

Задатак 1.

Корисник уноси своју лозинку преко тастатуре. Написати програм који проверава да ли лозинка има најмање 8 карактера и да ли се састоји искључиво од слова и бројева (без специјалних знакова). Исписати одговарајући True/False одговор.

Задатак 2.

Креирати листу са именима 4 града. Написати програм који сортира градове у обрнутом азбучном редоследу (descending) и штампа тако добијену листу.

Задатак 3.

Дата је торка cene = (150, 200, 350, 150, 400, 150). Написати програм који без промене торке броји колико пута се појављује цена 150 и штампа индекс њеног првог појављивања.

Задатак 4.

Написати програм који креира речник за један артикал у продавници (са кључевима "naziv", čena", "kolicina"). Програм треба да измени цену артикла на нову вредност (коју корисник уноси), дода нови пар "popust": 10и одштампа све кључеве и вредности паралелно.

Задатак 5.

Корисник преко тастатуре уноси реч дужу од 5 слова (нпр. "Programer"). Написати програм који узима ту реч и формира нови стринг који се састоји од прва два и последња два слова унете речи (у овом примеру резултат треба да буде "Prer"). Помоћ: Користите комбинацију позитивног и негативног исецања.

Задатак 6.

Дата је почетна листа оцена једног студента: ocene = [6, 8, 9, 7, 10, 6]. Написати програм који:

- Из листе уклања прву појаву оцене 6 помоћу методе remove().
- Избацује овесени елемент са индекса 2 (користећи pop()) и штампа вредност те избачене оцене.
- На крају додаје нову оцену 9 и штампа коначну дужину листе

Задатак 7.

На једном факултету, група студената A слуша курс из Python-а: python_studenti = {"Ana", "Milos", "Ivan", "Elena"}. Група B слуша курс из База података: baze_studenti = {"Petar", "Ana", "Elena", "Nikola"}. Написати програм који користи одговарајуће методе сетова како би пронашао и исписао:

- Студенте који слушају оба курса истовремено (пресек).
- Студенте који слушају само Пытхон, али не и Базе података (разлика).

Задатак 8.

Креирати угњеждени речник под називом imenikkоји садржи податке за две особе. Свака особа треба да има свој подречник са кључевима "telefon"и "grad". Особа 1: Кључ је "Marko", телефон је "064-111-222", град је "Beograd". Особа 2: Кључ је Šara", телефон је "065-333-444", град је "Nis". Програм треба да директно приступи и одштампа Сарин број телефона, а након тога да измени Марков град у "Novi Sad"и прикаже цео речник.

3 Задаци за вежбање

Задатак 1.

Написати програм који учитава реченицу са тастатуре, уклања празне просторе (белине) са почетка и краја те реченице, а затим сва појављивања речи "los"замењује речју ђobar". Коначан резултат исписати великим словима.

Задатак 2.

Корисник уноси име артикла, његову цену (реалан број) и количину (цео број). Формирати речник артикал са унетим подацима. Након тога, израчунати укупну вредност директно унутар једног f-стринга заокружену на две децимале и приказати је на екрану.

Задатак 3.

Написати програм који спаја два унета скупа (сета) А и Б у један заједнички скуп који садржи све елементе из оба скупа (унија). Након тога, проверити да ли се у том новом скупу налази број 5 и штампати одговарајућу логичку поруку.

Задатак 4.

Дати су подаци о једном аутомобилу у виду торке: ауто 1 (”Ауди”, ”A4”, 2018, Дизел”). Написати програм који врши ”отпакивање”(унпацкинг) ове торке у појединачне променљиве марка, модел и преостале податке који се пакују у листу под називом детаљи. Штампати вредности тих нових варијабли.

4 Литература/Корисни линкови

1. `https://imi.pmf.kg.ac.rs/moodle/course/view.php?id=327`

Датум ажурирања: 22.10.2026.